The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds

Taher A. Ghaleb
Trent University
Peterborough, Ontario, Canada
taherghaleb@trentu.ca

Daniel Alencar da Costa
University of Otago
Dunedin, New Zealand
danielcalencar@otago.ac.nz

Ying Zou
Queen's University
Kingston, Ontario, Canada
ying.zou@queensu.ca

Abstract

Continuous Integration (CI) provides early feedback by automatically building software, but long build durations can hinder developer productivity. CI services use caching to speed up builds by reusing infrequently changing artifacts, yet little is known about how caching is adopted in practice and what challenges it entails. In this paper, we conduct a large-scale empirical study of CI caching in Travis CI, analyzing 513,384 builds from 1,279 GitHub projects. We find that only 30% of projects adopt CI caching, and early adopters are typically more mature, with more dependencies, commits, and longer CI lifespans. To understand non-adoption, we submit pull requests enabling caching in non-adopting projects, and nearly half are accepted or merged. Developer feedback indicates that non- or late adoption mainly results from limited awareness of CI caching support. We further study cache maintenance and identify five common activities, performed by 24% of cache-enabled projects. While one-third of projects see substantial build-time reductions, cache uploads occur in 97% of builds, and 27% of projects contain stale cached artifacts. An analysis of reported caching issues shows developers mainly struggle with corrupted or outdated caches and request broader caching features. Overall, CI caching does not benefit all projects, requires ongoing maintenance, and is more complex in practice than many developers expect.

CCS Concepts

• **Software and its engineering → Software development process management**; *Software configuration management and version control systems*; *Software maintenance tools.*

Keywords

Continuous Integration (CI), Travis CI, CI caching; Empirical software engineering, Mining software repositories, Process mining

ACM Reference Format:
Taher A. Ghaleb, Daniel Alencar da Costa, and Ying Zou. 2026. The Promise and Reality of Continuous Integration Caching: An Empirical Study of Travis CI Builds. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

EASE 2026, Glasgow, Scotland, United Kingdom

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Continuous Integration (CI) allows software developers to integrate code changes into remote repositories, automatically generate builds, and detect errors early [19]. However, CI builds can be slow [2, 6, 23, 26], wasting resources and hindering other development activities. CI services provide caching for infrequently changing artifacts [57]. As reported by prior work, caching can speed up builds [17, 20, 23]. However, caching is not enabled by default; developers must configure which artifacts to cache. Yet, developers may (a) lack awareness of which artifacts change less frequently, (b) misuse caching configurations, and (c) invest time/effort maintaining the cache throughout the project's lifetime. Moreover, prior research has not established whether caching is always beneficial. As the Travis CI team states: "*There is pros and cons of using the cache. It is up to you [developers] to decide whether you want to use it*".[1] Although CI caching can speed up builds, the associated challenges (e.g., maintenance effort) and issues (e.g., unexpected failures) remain underexplored. There is also little empirical guidance on how developers should adopt caching carefully, how well caching is adopted and maintained in open source projects, and whether it benefits every project. Investigating current practices, challenges, and issues of CI caching can help developers set realistic expectations and avoid unnecessary overhead.

In this paper, we empirically study the adoption of CI caching in Travis CI, a cloud-based CI service. We analyze 513, 384 builds from 1, 279 GitHub projects using Travis CI. We investigate the adoption rate of CI caching to understand why some projects adopt it early or proactively, and why others do not adopt or delay adoption. We group the studied projects into five categories based on caching adoption and model differences among these categories using logistic regression. To gain further insights on caching adoption, we submit 85 pull requests to enable CI *cache* in projects that had never used caching, and we comment on pull requests/commits where CI caching was adopted proactively or belatedly, asking for justifications. To understand potential overhead, we also analyze cache maintenance activities, the impact of caching on CI builds, and commonly reported CI caching issues.

Research Questions (RQ_s). We study current CI caching practices and challenges across five research questions (RQs):

RQ_1: What urges software projects to adopt CI caching? CI caching can speed up builds, yet 70% of projects in our dataset do not use it, mainly due to lack of awareness of CI caching support. As a result, developers of these projects incur opportunity costs by disregarding the benefits of caching.

[1] travis-ci/travis-ci/issues/6396#issuecomment-240926751

RQ$_2$: How do developers maintain CI caching? Even with caching enabled, developers must regularly update caching configurations to match code changes. However, maintenance practices for CI caching are not well understood. By mining changes to CI caching configurations in our projects, we identify five common maintenance activity patterns. Still, most projects (76%) perform no maintenance after adoption. Developers should regularly maintain cached artifacts to preserve the benefits of CI caching.

RQ$_3$: To what extent does CI caching reduce the build duration? Prior research has paid little attention to whether CI caching consistently reduces build duration. We model the change in build time before and after adopting CI caching to assess its impact. Two-thirds of projects see no reduction, and 80% of those do not maintain CI caching regularly. Developers should not treat CI caching as a "*one size fits all*" solution.

RQ$_4$: How much overhead does CI caching introduce to builds? CI caching adds steps such as downloading and uploading caches, but their impact on build time is unclear. CI log analysis shows that cache uploads occur in 97% of builds and take six times longer than downloads. We find that frequent cache uploads stem from caching rapidly changing artifacts, such as installation logs. CI services should assist developers in monitoring CI logs for abnormal cache uploads and exclude unnecessary artifacts from the CI cache.

RQ$_5$: What issues do developers encounter with CI caching? Adopting CI caching can introduce problems such as occasional build failures. Identifying these problems helps developers avoid them and decide whether to adopt CI caching or not. In this RQ, we analyze caching-related issues reported to the Travis CI team. Our results show common issues with cache storage, where artifacts become corrupted or outdated. The CI team recommends regularly clearing the CI cache, which resolves most caching issues.

In summary, this paper makes the following contributions:

- An empirical study showing limited caching adoption in Travis CI and its association with project maturity.
- An intervention study showing that CI caching enabled via pull requests is accepted by nearly half of developers.
- A process-mining analysis revealing the ongoing maintenance required for CI caching.
- An impact analysis showing that adopting CI caching does not consistently reduce build duration.
- An analysis of issue reports identifying cache corruption and limited tool support as major challenges.

Paper organization. The rest of this paper is organized as follows. Section 2 presents background about CI. Section 3 describes the experimental setup of our study. Section 4 presents the results and findings of our studied RQs. Section 5 discusses the implications of our findings. Section 6 presents threats to the validity of our results. Section 7 presents the related literature on CI builds. Finally, Section 8 concludes the paper and outlines possible future work.

2 Background

Continuous Integration (CI). CI automates software builds and provides early feedback on code changes [19]. Builds are typically triggered by commits or pull requests, fetching the source code, installing dependencies, building the software, and running tests. CI builds pass if all steps succeed, otherwise they fail.

Travis CI. Travis CI[2] is a widely used cloud-based CI service [42]. Its build lifecycle consists of two main phases, `install` and `script`, plus an optional `deploy` phase. During `install`, the repository is cloned and dependencies installed; during `script`, code is built and tests run; `deploy` packages and releases software. Developers customize builds via `.travis.yml`, including machine specifications, timeouts, and CI features such as caching.

CI Caching. On December 5, 2013, Travis CI introduced caching for private repositories [54]. Support for public repositories followed on December 17, 2014, with the introduction of container-based infrastructure [55]. Finally, caching became available across all infrastructures on May 3, 2016 [56]. Developers enable caching with the `cache:` directive, typically for dependencies [57], and use `before_cache:` to delete temporary files, such as logs. Cached artifacts are uploaded after a build, reused in later builds, and only changed files are re-uploaded. Builds proceed even if cache operations (downloads or uploads) fail, and caches can be invalidated via the API or on the web. Each CI job has its own cache. Other popular CI services, such as GitHub Actions and CircleCI, provide similar build customization and caching mechanisms, where developers can define cache contents, control storage/retrieval, and clear outdated caches. Given the shared principles, we expect our findings on Travis CI caching to have broader relevance, offering guidance for optimizing caching strategies in other CI environments.

3 Experimental Setup

This section describes the experimental setup of our study and how we collect and prepare the data for our RQs.

3.1 Data Collection

Figure 1 shows an overview of our study, which uses data from TravisTorrent [7], a common dataset for CI build research [6, 39, 41]. TravisTorrent includes builds from 1,283 projects: 886 Ruby, 393 Java, and 4 JavaScript. We exclude the four JavaScript projects, since they are not a representative sample of that language. Although the last build in our dataset was triggered on *Aug* 31, 2016, we further verified in 2021 that all projects remain active and maintained by examining their commit history and issue activity. We also interacted with developers from both caching adopters and non-adopters by submitting pull requests to non-adopters and commenting on commits where caching was enabled, receiving responses from over half of the contacted projects. Each TravisTorrent project includes builds from multiple branches. We consider only the default (e.g., *main* or *master*) branches, since they are more active and contain more builds. We identify each repository's default branch using the GitHub API.[3] This yields 513,384 builds. For each build, we collect the CI build job logs using the Travis API.[4]

3.2 Data Processing

This section explains how we process data from the 1,279 projects.

[2] https://travis-ci.com
[3] https://docs.github.com
[4] https://docs.travis-ci.com/api

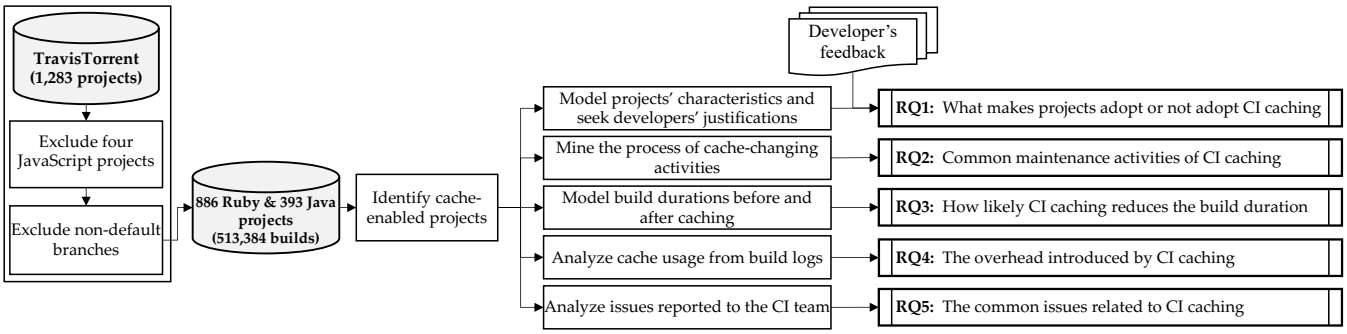

Figure 1: Overview of our study

3.2.1 ***Computing project-level metrics.*** Our study conducts project-level analyses of CI caching adoption. Specifically, we:

- detect whether a project adopts CI caching by analyzing its `.travis.yml` configuration file;
- determine when a project adopts CI caching by locating the first commit that enables it; and
- identify characteristics of projects that adopt or do not adopt CI caching by analyzing each project's last build before adoption.

We examine how project characteristics are associated with CI caching adoption. For each characteristic, we compute 13 project-level metrics (see Table 1) from builds triggered before caching adoption. Our data and scripts are publicly available in our replication package [24].

- *Programming Language:* We analyze whether CI caching adoption is associated with a project's programming language, since caching needs can vary by language.
- *Code Maturity:* We investigate whether code maturity (e.g., SLOC, test density, number of dependencies) is related to CI caching adoption.
- *Development Activity:* We study whether more active projects (e.g., with more frequent commits) adopt CI caching differently from less active projects.
- *CI Activity:* We analyze whether CI activity (e.g., build configurations and durations) is associated with CI caching adoption.

3.2.2 ***Performing correlation and redundancy analyses.*** We use project-level metrics as independent variables in our logistic regression models (see Section 4.1:RQ_1). Following Harrell's regression modeling guidelines [30], we first remove highly correlated variables, which can adversely affect regression models [16]. We apply Spearman rank ρ clustering [50] (via the `varclus` function in the `rms` *R* package [34]) to identify highly correlated variables. For each pair with $|\rho| > 0.7$, we retain one variable and discard the other. By the principle of parsimony, simple variables are preferred [60]. Since all variables are similarly simple to compute, we keep those more informative about development and build processes. For instance, we exclude '*# of builds*', which is highly correlated with '*# of commits per lifetime*'. We then perform redundancy analysis on the remaining variables, as redundant predictors can also distort regression models [30]. Using the `redun` function from the `rms` *R* package, we search for variables that can be modeled by others with $R^2 \geq 0.9$ and find no redundant variables.

4 Empirical Analysis and Results

This section discusses RQ motivation, approaches, and findings.

4.1 RQ_1: What urges software projects to adopt CI caching?

4.1.1 ***Motivation.*** CI caching lets builds download software artifacts from the CI server instead of reinstalling them from original sources. Despite its potential speedup, some projects do not adopt CI caching. In this RQ, we (i) measure the proportion of projects that adopt CI caching and (ii) analyze factors associated with adopting, not adopting, or delaying its adoption.

4.1.2 ***Approach.*** We identify projects that adopt CI caching by analyzing their build configuration files (i.e., `.travis.yml`) and checking for the '`cache:`' directive. For each project using CI caching, we locate the first commit that introduces the '`cache:`' directive and record this as the enabling date. We then compute the number of days between this date and the announcement of CI caching for open source projects (*Dec* 17, 2014). Figure 2 shows the distribution of this *adoption delay*. We use the 1^{st} quantile (41 days) and 3^{rd} quantile (12.2 months) of the delay as thresholds to classify projects as *early*, *ordinary*, or *late* adopters: (i) *early* if the delay is between 0 and 41 days, (ii) *ordinary* if it is between 41 days and 12.2 months, and (iii) *late* if it is between 12.2 and 20 months. Based on these thresholds, we group the studied projects into five categories.

- ***Non-adopters:*** projects that never adopt CI caching in their builds (899 projects).
- ***Proactive adopters:*** projects that adopt CI caching before it was officially available (76 projects).
- ***Early adopters:*** projects that adopt CI caching within 41 days of being officially available (77 projects).
- ***Ordinary adopters:*** projects that adopt CI caching within a year of being officially available (151 projects).
- ***Late adopters:*** projects that adopt CI caching after a year of being officially available (76 projects).

To identify factors behind none, early, and late CI caching adoption, we model differences among the five project categories using project-level metrics. We employ Generalized Linear Models (GLMs) with logistic regression to analyze which factors relate to delayed or absent CI caching adoption. We fit five GLMs (one per category) using the `glm` function from the `stats`[5] R package. Each model compares projects in a given category (e.g., *early* adopters) to all others,

[5]https://www.rdocumentation.org/packages/stats

Table 1: Description of project-level metrics. All metrics are computed before a project adopts CI caching

Project characteristic	Project-level metric	Description
Programming Language	Language	The GitHub dominant programming language of a project
Code Maturity	Size (SLOC)	Number of source lines of code of a project
	Dependencies	Number of dependencies (i.e., libraries, packages, or gems) a project relies on
	Test density	Median number of test cases per 1,000 SLOC of a project
Development Activity	# of commits per lifetime	Ratio of the number of commits per project age
	Growth rate	Ratio of the relative increase/decrease (i.e., delta) in the lines of code
	# of developers	Number of unique developers contributed to a project
CI Activity	CI lifespan	Time (in terms of days) since a project adopted Travis CI
	# of builds	Number of CI builds triggered by a project
	Building frequency	Frequency (in terms of days) of triggering builds in a project
	Configuration ratio	Ratio of commits that change the build configuration file (`.travis.yml` per project lifetime
	Build environments	Number of unique integration environments used as build jobs in a project
	Build duration	Median build duration of a project

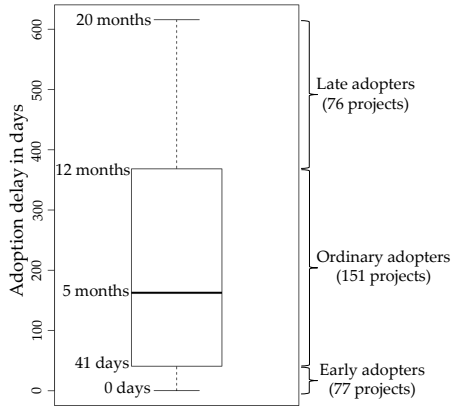

Figure 2: Boxplot of the CI caching adoption delay

with a logical dependent variable: *TRUE* if a project belongs to the category and *FALSE* otherwise. The models use 13 project-level metrics (see Section 3.2) as independent variables.

Seeking feedback from caching non-adopters. Developers may skip CI caching simply due to unawareness or carelessness. To investigate this, we focused on projects that never use caching on Travis CI. Of 899 such projects, we manually fork 160 that (i) still had not adopted caching since data collection and (ii) had recent commits (in 2020 or later). We trigger CI builds on these projects using the latest commit on their default branches (without further configuration changes), then exclude 71 projects whose builds fail and four having no dependencies. For the remaining 85 projects, we push a commit enabling dependency caching. After confirming that all builds triggered by our commits are *passing*, we submit a pull request to each project asking why CI caching was not enabled.

Seeking feedback from proactive and late caching adopters. To understand why projects adopted CI caching *proactively* or *late*, we commented on commits or pull requests that enabled the CI *cache* in the past. For *proactive* adopters, we asked developers from 74 projects why they enabled caching before it was available for open source projects. For *late* adopters, we asked developers from 71 projects about the reasons for their delayed adoption. We then waited several months for responses.

4.1.3 ***Findings.*** Table 2 shows the results obtained from our logistic regression models of the differences between the studied projects in adopting CI caching. In Table 3, we show a summary of the responses we obtain from developers to our submitted pull requests and comments to none/proactive/late caching adopters. We expected a quick and high acceptance rate for our pull requests, regardless of the prospective speed up CI caching might bring to CI builds, since CI caching is less likely to cause issues with CI builds. However, we observe that nearly half (48%, 41/85) of our pull requests were approved/merged by project maintainers, with 25% (21/85) closed, 5% (4/85) remaining open with responses, and 22% (19/85) still open without any response. Some pull requests took several months before being accepted or closed. Among the merged pull requests, 22% (9/41) provided no response, while 20% (8/41) explicitly stated they are not aware of the caching feature. A quarter (25%, 21/85) were closed, most commonly due to projects migrating (12/21) to other CI services (e.g., GitHub Actions[6]) [11, 32].

Observation 1.1. ***Over two-thirds of the studied projects do not adopt CI caching.*** Despite being available to every GitHub project, we find that CI caching has been adopted by only 30% (380 out of 1,279) of the projects in our dataset. We observe from Figure 2 that about two-thirds of the caching adopters took over a month before enabling caching in their builds. Nevertheless, 20% of the caching-adopting projects adopted CI caching proactively. Based on the responses received from developers, 39% of such projects were given an opportunity to use CI caching unofficially (i.e., a preview), whereas 39% of the projects were configured to use caching configuration by mistake (e.g., misunderstanding or a *copy/paste* from other projects). In addition, we observe that projects that do not adopt CI caching have significantly shorter build durations, fewer developers, and fewer build configurations. This result hints that projects that do not adopt CI caching are not as active or mature enough to seek the benefits of CI caching.

Our analysis of the responses received from projects that do not adopt CI caching reveals several reasons. The most common reason is that developers perceive no need or benefit from caching (21%, 14/66), followed by projects migrating to other CI services (18%, 12/66). Some developers are unaware of the caching feature (14%, 9/66), and some did not respond (14%, 9/66). Interestingly, some developers assumed caching was enabled by default (3%, 2/66),

[6]https://github.com/features/actions

Table 2: Modeling the differences between projects based on the caching adoption delay

Project characteristic	Project-level metric	*None/Proactive*		*None/Early*		*None/Ordinary*		*None/Late*		*Proactive/Early*	
		χ^2%	Signf.+	χ^2%	Signf.	χ^2%	Signf.	χ^2%	Signf.	χ^2%	Signf.
Language	Ruby	1.26		14.86	* (↗)	21.58	* (↘)	0.02		15.89	* (↗)
Code Maturity	Size (SLOC)	12.8		27.13	*** (↗)	0.53		0.81		6.93	
	Test density	0.46		0.09		21.56	* (↗)	46.71	** (↗)	2.04	
	Dependencies	6.85		13.38	* (↗)	5.56		2.35		7.88	
Development Activity	# of commits per lifetime	0.83		1.19		13.4	.	0.48		1.17	
	Growth rate	5.26		0.48		0.03		1.92		0.12	
	# of developers	12.83		6.46	.	0.11		17.25		25.2	** (↘)
CI Activity	CI lifespan	15.16	.	3.78		10.87	.	14.09		9.11	
	Building frequency	10.59		5.3		1.05		4.15		7.16	
	Configuration ratio	5.81		5.19		2.71		0.98		0.72	
	Build environments	9.11		5.72		0.42		2.96		22.89	* (↗)
	Build duration	19.02	* (↗)	16.41	** (↗)	22.19	* (↗)	8.27		0.89	

		Proactive/Ordinary		*Proactive/Late*		*Early/Ordinary*		*Early/Late*		*Ordinary/Late*	
		χ^2%	Signf.+	χ^2%	Signf.	χ^2%	Signf.	χ^2%	Signf.	χ^2%	Signf.
Language	Ruby	0.23		18.42		22.3	* (↘)	5.82		23.36	.
Code Maturity	Size (SLOC)	10.08		1.19		13.9	.	7.25		0	
	Test density	12.73		19.52		0		2.49		22.04	.
	Dependencies	0.08		8.5		12.21	.	24.02	* (↘)	14.41	
Development Activity	# of commits per lifetime	1.31		0.08		3.28		0.45		3.8	
	Growth rate	23.03		38.42	* (↘)	2.86		2.47		5.59	
	# of developers	24.67		0.06		17.58	* (↗)	33.97	** (↗)	14.69	
CI Activity	CI lifespan	0.1		1.58		5		0.74		1.25	
	Building frequency	10.29		1.07		4.85		10.94		7.81	
	Configuration ratio	0.09		0.01		2.06		0.6		0.34	
	Build environments	16.44		9.01		12.1	.	10.12		1.38	
	Build duration	0.95		2.14		3.86		1.12		5.31	

+Significance codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Table 3: Developer responses regarding CI caching adoption

Caching adoption	Action	# (%) responses	#	Response
Non-adopters (pull requests)	*Approved/Merged*	41/85 (48%)	9/41	No response given
			8/41	Not aware of caching feature
			6/41	No perceived need or benefit
			5/41	Appreciation only
			4/41	Oversight or forgot
			4/41	No particular reason or don't remember
			2/41	Assumed caching enabled by default
			2/41	Unclear or uncertain
			1/41	Technical handling issue
	Open (with response)	4/85 (5%)	3/4	No perceived benefit
			1/4	Introduces complexity
	Open (no response)	19/85 (22%)	19/19	No response
	Closed	21/85 (25%)	12/21	Migrating to other CI services
			5/21	No perceived benefit
			2/21	Unknown reason
			1/21	Troublesome or increases workload
			1/21	Not aware of caching feature
Proactive adopters (comments)		22/74 (30%)	9/22	Caching was available unofficially (pre-release)
			5/22	Developers' misunderstanding
			4/22	Mirrored or copied from another project
			3/22	Developers cannot recall / no specific reason
			2/22	Travis CI is no longer used
Late adopters (comments)		40/71 (56%)	21/40	Unaware of the caching support
			12/40	Developers are busy or have other priorities
			7/40	Developers cannot recall / no specific reason

indicating misconceptions about CI configuration, while others admitted it was an oversight or they forgot to enable it (6%, 4/66). Another group had no specific reason or could not remember why caching was not enabled (6%, 4/66). In addition, 22% (19/85) of submitted pull requests received no response. Still, caching support may attract developers back to Travis CI after prior disabling.[7] These results suggest CI services should communicate updates about performance-improving features like caching more effectively.

Observation 1.2. ***Projects with shorter build durations, smaller development teams, and fewer build configurations are unlikely to adopt CI caching.*** We observe that projects that do not adopt CI caching have significantly shorter build durations than those that do, consistent with developers stating that caching is unnecessary when builds are already fast.[8,9,10] We also observe that Non-adopting projects also have fewer developers (median = 10) than adopting projects (median = 14). These findings suggest that (a) larger teams are more likely to be aware of CI caching and (b) the need for caching grows with longer build durations, as more developers are expected to wait for builds to finish.

Observation 1.3. ***Projects with more commits and dependencies are more likely to early adopt CI caching.*** Caching dependency binaries maximizes the benefit of CI caching. The studied projects have a median of 39 dependencies, and our models show that dependency count is significantly associated with early CI caching adoption: each additional ten dependencies increase the odds of earlier adoption by 4%. Moreover, projects that do not adopt CI caching have 29% fewer dependencies than those that do. This suggests that developers of projects with few dependencies expect little build-speed improvement. For example, developers of the `test-unit/test-unit` project, which has ten dependencies, reported that CI caching brought negligible speedup.[11] It is worth noting that caching can also be applied to artifacts other than dependencies (e.g., arbitrary directories[12]) that change infrequently.

Observation 1.4. ***Late CI caching adoption is strongly associated with more build configuration activities.*** We observe that projects that have more build configuration activity tend to adopt CI caching later. For example, the rubinius/rubinius project made over 100 build configuration changes during 5 years of Travis CI usage, but adopted caching only about 1.5 years after it became available for open source projects. Before adopting caching, developers made many build configuration changes, sometimes to speed up builds (e.g., by removing build jobs [23]). Our analysis of commits and pull requests enabling caching indicates that late adoption was mainly due to (a) lack of awareness of Travis CI caching support (54%) or (b) higher-priority project tasks (31%).

RQ_1 Summary. Only 30% of projects adopt CI caching. Active, mature projects adopt it early, but misconceptions and lack of awareness lead others to not adopt or to delay adoption.

[7] ccw-ide/ccw/commit/032e253
[8] apache/storm/pull/902
[9] davidmoten/rxjava-jdbc/pull/96
[10] Esri/geometry-api-java/pull/283
[11] test-unit/test-unit/pull/123
[12] https://docs.travis-ci.com/user/caching/#arbitrary-directories

4.2 RQ_2: How do developers maintain CI caching?

4.2.1 ***Motivation.*** *RQ_1* shows that most projects do not adopt CI caching. Even when caching is enabled, developers need to regularly update the configuration to match code changes (e.g., adding or removing dependencies). Although such maintenance is important, it creates time and effort overhead similar to other build maintenance activities [40]. In this RQ, we investigate the historical changes to caching configurations to identify common patterns of CI cache-changing activities.

4.2.2 ***Approach.*** We identify cache-related commits that (a) modify caching configuration (‘`cache`‘ or ‘`before_cache`‘) in `.travis.yml` or (b) mention ‘*cache*‘ or ‘*caching*‘ in their messages. This yields 763 commits from 380 projects using CI caching (median: one cache-changing commit per project). After manually excluding 180 commits unrelated to CI caching (e.g., formatting or build refactoring), we label the remaining 583 commits by type of CI caching activity.

We generate a chronologically sorted list of cache-changing activities for each project. To identify patterns in these activities, we convert them into event logs and analyze them with the process mining framework ProM 6 [59]. Process mining allows to uncover the underlying process of caching activities. Using ProM 6, we derive common maintenance patterns, event timelines, and state chart diagrams for the cache-changing events. We then apply the *Fuzzy Miner* plugin of ProM 6 to abstract the mined process model and produce a fuzzy graph [29]. We call the first and last cache-changing activities of each project the *first event* and *last event*, respectively. Finally, we extract the artifact types cached by the `cache` directive to determine what different projects cache in common.

4.2.3 ***Findings.***

Observation 2.1. ***Only*** 24% ***of caching adopters maintain their CI caching configuration.*** Our analysis of cache-changing commits shows that most projects (76%) enable CI caching once and never update it, confirming prior work [5] in which over 80% of configuration files remain unchanged after creation. We also find that projects with a one-time caching configuration typically cache all dependencies managed by tools such as *Bundler*, *Maven*, or *Gradle*. Although this lets builds install only newly added dependencies, it can cause over-caching.[13] As a result, dependencies no longer used by a project (e.g., outdated dependency versions) may remain stale in the CI cache. If a CI cache is unused for some time, Travis CI clears it automatically.[14] However, caching frequently updated dependencies can delay cache expiration and cause build problems.[15] Travis CI allows developers to clear caches manually, but we find no evidence that they do so, as cache storage activities are not logged. Thus, developers should avoid over-caching and consider clearing CI caches regularly.

Observation 2.2. ***Developers perform five distinct maintenance activities when updating CI caching configurations.*** Table 4 lists cache-related activities that developers perform (at the commit level). Among the 24% of projects that maintain CI caching, we observe a median of three caching-related activities per project. After initially enabling CI caching, the most common activity is updating

[13] https://eng.localytics.com/best-practices-and-common-mistakes-with-travis-ci
[14] https://docs.travis-ci.com/user/caching/#caches-expiration
[15] travis-ci/travis-ci/issues/3758

(adding/removing) cached directories. While updating cached directories can improve build performance, directories previously stored in the cache are not removed automatically. Moreover, only 14% of cache-maintaining projects exclude unnecessary artifacts from the cache. We also find that 50% of cache-changing activities are submitted through pull requests or in response to reported issues, suggesting that (a) developers maintain cache configurations on separate branches before merging into the default branch, or (b) external contributors propose cache changes. Developers should therefore update cached directories carefully and exclude frequently changing artifacts before uploading caches to the CI server.

Table 4: Statistics of maintenance activities of CI caching

Maintenance Activity	Freq.	First event	Last event
Enable cache	68.2%	99.2%	80.8%
Update cached directories	15.0%	0%	7.9%
Disable cache	7.4%	0.8%	5.3%
Update cache configuration	5.6%	0%	3.2%
Exclude artifacts from cache	2.9%	0%	2.1%
Update directories & Exclude artifacts	0.9%	0%	0.8%

Observation 2.3. ***The process of changing the CI cache configuration is unsystematic and repetitive.*** Figure 3 shows a fuzzy graph of cache-changing activities in the studied projects. Edge thickness and numeric labels indicate the percentage of events that transition from one activity to another. We observe that cache-related events are often circular (e.g., enable-disable-enable) or repetitive (e.g., occurring multiple times in sequence). As Figure 3 depicts, developers frequently disable CI caching and then re-enable it in 85% of the cases, suggesting a trial-and-error process. For example, we find that developers may disable caching temporarily until a build failure is resolved.[16] In addition, we observe that developers update the cached directories repeatedly (up to eight times per project), suggesting difficulty deciding what to cache. For example, the cached directories of the 'SonarSource/Sonarqube' project[17] were updated eight times, adding artifacts (e.g., `node_modules` and `.sonar/cache`) and later removing them. About half of the caching changes occur after a median of 31 days since the previous change, and cache updates appear irregularly throughout the project lifetime (see the timeline of cache-changing events in our replication package). These findings suggest that (i) developers should closely monitor changes affecting the CI cache, (ii) researchers and tool builders should create tools that track historical code changes to identify frequently changing artifacts (and should be excluded) or infrequently (and should be cached), and (iii) Travis CI should use CI logs to recommend candidate artifacts for caching.

Observation 2.4. ***The majority (67%) of the projects cache all software dependencies.*** The Travis CI documentation provides simple caching examples for various languages,[18] which developers may interpret as best practices. Our analysis of the `cache` directive shows that 80% of Ruby builds and 45% of Java builds cache all dependencies. Other projects either customize caching from the start or initially cache an entire directory and later restrict it to subdirectories. For example, the `cloudfoundry/uaa`[19] project first cached

[16] ms-ati/docile/commit/8c4b9a8
[17] https://github.com/SonarSource/sonarqube
[18] https://config.travis-ci.com/ref/job/cache
[19] cloudfoundry/uaa/commit/53249a7

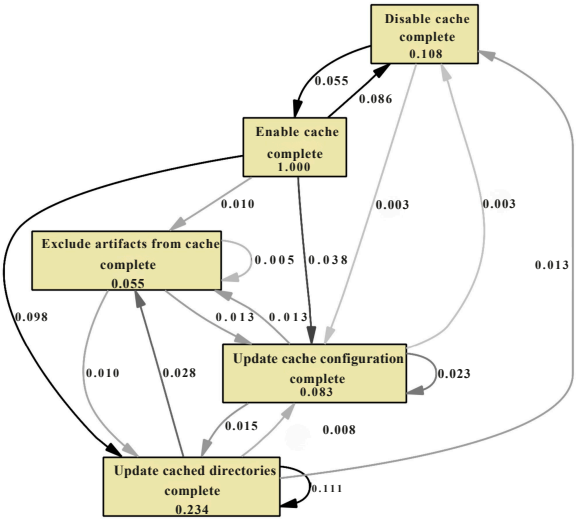

Figure 3: A Fuzzy Graph of cache-changing activities

all artifacts in `$HOME/.m2` (matching the documentation example), then limited caching to `$HOME/.m2/repository`. Dependency managers such as *Bundler* and *Maven* generate log files during compilation or installation. Because these files change in almost every build, they can trigger uploads of the entire cache to the CI server [57]. We observe that only a negligible number (4%) of the studied projects pre-process artifacts (e.g., exclude directories/files) before uploading the cache. For example, the `formtastic/formtastic`[20] project initially cached all artifacts in `vendor/bundle`, but later excluded generated logs (i.e., `gem_make.out, mkmf.log`) from the cache. We therefore encourage developers to carefully identify and exclude frequently changing artifacts from the CI cache.

RQ_2 Summary. The CI cache should be maintained to handle software changes over time. However, only 24% of projects regularly maintain their CI cache. Developers should regularly update cached artifacts to continuously benefit from CI caching.

4.3 RQ_3: To what extent does CI caching reduce the build duration?

4.3.1 ***Motivation.*** The number and type of cached artifacts can determine how much CI caching speeds up builds. However, prior work has not examined whether CI caching actually reduces build duration in practice. In this RQ, we investigate whether current CI caching practices are associated with shorter build durations.

4.3.2 ***Approach.*** We use Regression Discontinuity Design (RDD) [12], a quasi-experimental pretest-posttest design, to model changes in build duration before and after adopting CI caching. In RDD, an *intervention* is assigned based on a *cutoff*, allowing estimation of causal effects [33]. RDD assumes the pre-intervention trend would continue unchanged without the intervention. Here, the intervention is CI caching adoption, and the cutoff is the time at which caching is enabled. We apply RDD to assess whether CI caching adoption is associated with build duration. For each

[20] formtastic/formtastic/commit/2c13f86

project, we split builds into pre- and post-adoption and include all the builds before and after the CI adoption. We use the `RDestimate` function from the `rdd` R package.[21] Projects where `RDestimate` fails (e.g., due to insufficient data around adoption) are excluded. Overall, it successfully models the impact of CI caching on build duration in 75% of projects. We consider CI caching to affect build duration when the RDD estimate is significant ($p\text{-}value \leq 0.05$). Each RDD model yields a Local Average Treatment Effect (LATE), the average difference between expected build durations before and after adoption. We use LATE to determine whether CI caching is associated with longer (positive LATE) or shorter (negative LATE) build durations. For instance, a negative LATE for a project indicates that CI caching is associated with reduced build duration.

4.3.3 ***Findings.*** **Observation 3.1.** ***Only a third of the projects significantly reduce build duration after adopting CI caching.*** Our project-level RDD models show that build duration becomes significantly shorter following CI caching adoption in only 33% of projects, while it becomes significantly longer in 17% and shows no significant change in the remaining 50%. To explain this, we analyze projects with either a significant increase or no significant change in build duration after adopting caching. We observe that in 30–60% of projects where CI caching is not associated with shorter builds, there are significant increases in (a) source lines of code, (b) build jobs, and (c) test density (details are in our replication package). Thus, beyond CI caching, developers should consider other alternatives to speed up builds (e.g., parallelizing test suites [53] or removing duplicate tests [23]). These concurrent increases suggest that caching alone cannot compensate for growing build complexity, which explains why only a third of projects benefit from it.

Observation 3.2. ***One-time caching configuration is strongly associated with decreased or insignificant changes in build durations.*** In 80% of projects with insignificant changes or significant increases in build duration, we observe that developers configured CI caching only once (i.e., without further maintenance). CI build log analysis shows that the *script* phase consistently dominates the overall build duration. Thus, even when caching works as intended, it is unlikely to help much with those projects that do not heavily use external dependencies.[22] Overall, CI caching should not be treated as a "one-size-fits-all" solution.

RQ_3 Summary. CI caching is not a "*one size fits all*" solution, as two-thirds of projects see little or negative impact on build duration. Developers should regularly maintain CI caches to preserve benefits and prevent issues.

4.4 RQ_4: How much overhead does CI caching introduce to builds?

4.4.1 ***Motivation.*** Caching can improve CI build performance, but processing the cache (uploading, downloading, and storing) also adds overhead. Understanding this overhead helps developers decide whether caching is worthwhile. In this RQ, we analyze how CI builds process caches.

4.4.2 ***Approach.*** We automatically analyze 611,501 logs of CI build jobs in which caching is enabled. We identify dependencies downloaded from the CI cache (marked '`using`') and those missing from the cache (marked '`installing`') during dependency installation. We measure the time spent for downloading and uploading the cache and installing uncached dependencies. We also detect cached but unused dependencies by comparing the dependencies downloaded from the CI cache in build x_n with those used in the subsequent build x_{n+1}. We compute cache miss rates as the percentage of cached dependencies that are not used by future builds.

4.4.3 ***Findings.***

Observation 4.1. ***CI builds perform cache uploads very frequently.*** Our analysis of CI logs shows that 97% of builds upload a cache, indicating that caches often include artifacts that change almost every run. Uploading the cache takes a median of 12 seconds (six times slower than downloading), while installing new dependencies takes 8.5 seconds (3.25× slower than downloading). Longer cache-upload times increase the risk of caching unnecessary files, such as build logs.[23] Thus, caching can add overhead if misused. Therefore, CI services should provide cache-aware tooling [4] to help developers detect abnormal cache uploads from verbose CI logs and configure their builds to exclude unnecessary files before uploading the CI cache.

Observation 4.2. ***One-third of the projects have a cache miss rate of*** 33%**.** Our analysis of CI logs reveals that two-thirds of the projects have a zero cache miss rate. However, a fully hit CI cache may indicate outdated dependencies, making these projects more vulnerable to security issues [45]. Kula et al. [38] similarly report that over 80% of dependency-heavy projects do not update their dependencies. In contrast, one-third of the projects in our dataset frequently update dependencies, causing new or updated dependencies to be installed in most builds. While updating dependencies is good practice, developers should carefully identify the most frequently updated dependencies and exclude them from the CI cache. Otherwise, builds may be delayed because (a) older dependency versions are repeatedly loaded from the cache, and (b) newer versions are installed and then re-uploaded to the cache.

Observation 4.3. 27% ***of the studied projects have unused dependencies in the CI cache***. Caching frequently updated dependencies can leave outdated versions in the CI cache. In 73% of the projects, all cached dependencies are used in builds, suggesting these projects either cache only stable dependencies or manually remove unused ones. However, in 27% of the projects, dependencies used by earlier builds are not used later, implying that cache size can grow continuously if unused artifacts are not removed.[24] In fact, developers of two projects in our dataset cite stale cached dependencies as a reason for not adopting CI caching.[25,26] Beyond unnecessary delay, unused cached dependencies can cause storage issues, especially since a repository may keep separate CI caches for each branch and language/compiler version.

[21] https://search.r-project.org/CRAN/refmans/rdd/html/RDestimate.html
[22] FasterXML/jackson-core/pull/682
[23] twitter-archive/commons/commit/146de30
[24] Cockatrice/Cockatrice/issues/3781
[25] FasterXML/jackson-core/pull/682
[26] airlift/slice/pull/139

RQ_4 Summary. CI caches are uploaded in 97% of builds, and 27% include unused artifacts. Developers should carefully choose what to cache and regularly remove stale cached dependencies.

4.5 RQ_5: What issues do developers encounter with CI caching?

4.5.1 ***Motivation.*** CI caching requires regular maintenance, yet prior work has largely overlooked the related issues developers face. Understanding these problems and their resolutions helps developers better leverage caching. In this RQ, we investigate issues that arise when projects adopt CI caching.

4.5.2 ***Approach.*** We crawl GitHub for caching-related issues reported to the Travis CI team on or before *Aug* 31, 2016 (the last build-triggering date in our dataset). Using '*cache*' and '*caching*' as keywords, we collect 373 caching-related issues[27] from the Travis CI repository. These issues reflect common problems projects face when adopting Travis CI caching. When developers encounter a caching problem, they may comment on an existing issue instead of opening a new one.[28] Two co-authors manually analyze the GitHub issues. Using open coding [13], we identify common caching issues. To resolve coding disagreements, we apply negotiated agreement [10]: the two co-authors collaborate on coding a sample of 50 issues, group the resulting codes into categories, then code another 50 issues to check for new codes, repeating this process until no new codes are identified.

4.5.3 ***Findings.***

Observation 5.1. ***Outdated or corrupted caches are the most common CI caching issues.*** Cache upload/download errors rarely cause build failures, but developers frequently report failures when CI caching is enabled. Our analysis of cache-related issues reported to Travis CI shows that most failures are due to (i) corrupted caches,[29] (ii) caches not updating dependency versions,[30] or (iii) multiple jobs or branches overwriting a shared cache.[31] The Travis CI team replies with clarification questions or requests for supporting materials, and many caching problems are fixed in later releases.[32] When the root cause is unclear, they often suggest manually clearing the cache on the CI server,[33] which resolves most (62%) of the analyzed issues based on developer feedback. Details about common cache issues and the corresponding responses from the Travis CI team are available in our replication package [24].

Observation 5.2. ***Caching is a continuously evolving CI feature.*** Our manual analysis of CI caching issues reveals that caching keeps evolving, as the Travis CI team repeatedly updates the feature in response to developer requests. For example, developers often request caching support for specific environments (e.g., `macOS`[34]) or a simple directive to cache specific artifacts (e.g., `ccache`) instead of manually listing directories. The Travis CI team frequently agrees (e.g., promising support in future releases).[35] In other cases, team members propose workarounds[36] or disagree with the reported concerns.[37] Thus, developers should stay current with CI caching capabilities, which may already address previously unsolved issues.

RQ_5 Summary. Developers of projects that adopt CI caching often report cache corruption (e.g., from incompatible dependency versions) and request better support for specific caching directives (e.g., `ccache`). Caching capabilities are improving.

5 Implications

For project maintainers:

What else to cache? As RQ_2 shows, caching is mostly limited to dependencies. However, other artifacts can benefit from caching. For example, test-generated static data can significantly increase build duration [6, 23], so developers could cache such artifacts (e.g., test input data [37] or search results[38]). For Java projects, incremental compilation could cache previously compiled class files to avoid unnecessary recompilation. While Travis CI supports this for C/C++ via `ccache`, we found no equivalent for Java.

No pain, no gain! Our RQ_2 process mining of CI cache-changing commits reveals that developers must actively maintain caches as projects evolve. In 80% of projects that did not achieve significant build-time reductions, caches were not maintained, indicating that unmanaged caching is largely ineffective. While maintenance requires effort [40], continuously monitoring CI practices is important to sustain their benefits [49]. Our results suggest that even minimal monitoring helps preserve caching benefits.

For CI services and tooling:

One caching configuration may not fit all. RQ_3 shows that caching large files often fails to reduce build duration, and developers struggle to identify suitable directories. CI services should help (a) detect artifacts worth caching [20] and (b) identify stale cached artifacts so developers can better tune caching strategies. Developers should also periodically update cached dependencies to prevent dependency staleness and security risks [38].

Semantic linters for inefficient caching. As RQ_4 reports, developers tend to frequently cache directories that change often or provide minimal speed-up (e.g., *JDK* packages). CI services should offer semantic linters that flag inefficient caching, beyond simple syntax checks [8, 62, 67].

Simplifying selective caching. As per RQ_5, excluding sub-directories is complex and error-prone: developers must either list all other sub-directories in the `cache` directive or use `before_cache` to remove unwanted ones. CI services should support more flexible approaches (e.g., wildcards or regex) to reduce configuration effort and errors.

Practical, context-aware documentation. RQ_2 shows that two-thirds of projects follow sample caching configurations (e.g., Ruby's *bundler*), often caching all dependencies and limiting benefits. CI services could leverage historical build data to automatically suggest caching strategies based on documentation [27], such as selectively caching infrequently changing dependencies.

[27] travis-ci/travis-ci/issues
[28] travis-ci/travis-ci/issues/4393
[29] travis-ci/travis-ci/issues/6396
[30] travis-ci/travis-ci/issues/3758
[31] travis-ci/travis-ci/issues/4657
[32] travis-ci/travis-ci/issues/4393#issuecomment-219776423
[33] travis-ci/travis-ci/issues/3758#issuecomment-106989375
[34] travis-ci/travis-ci/issues/4869
[35] travis-ci/travis-ci/issues/4997#issuecomment-216615467
[36] travis-ci/travis-ci/issues/7456#issuecomment-296505058
[37] travis-ci/travis-ci/issues/4191
[38] thredded/thredded/commit/2f04819

Raising awareness of CI caching. RQ_1 reveals that 70% of projects do not adopt caching, mainly due to ignorance or mistakenly assuming it is enabled by default. CI services should (a) recommend caching at adoption time and (b) provide estimated time savings based on build history. With nearly half of our caching-enabling PRs accepted, CI services could further promote adoption (e.g., using bots or AI agents [22]) to automatically detect eligible projects and enable caching.

While our findings are based on Travis CI, other CI services like GitHub Actions share similar principles (e.g., manual caching setup) [28], thus our implications likely extend beyond Travis CI.

6 Threats to Validity

Construct Validity We rely on data from TravisTorrent and cloned Git repositories. Errors in computing metrics could affect results, but we carefully filtered and tested the data to reduce them. To identify CI caching issues, we used key terms likely to appear in relevant issues. This may retrieve unrelated issues; to address this, two co-authors collaboratively performed open coding to exclude them. Our findings come from exploratory models and manual analyses of real project data (data and scripts are publicly available [24]). Future work could survey developers to better understand their reasons for adopting or maintaining CI caching.

Internal Validity We use project-level metrics to analyze factors associated with (a) CI caching adoption delay and (b) build-duration changes, yet these metrics may not be exhaustive enough. For correlated metrics, the choice of which to retain may affect the models; to ensure reproducibility, all such choices are explicitly documented. Moreover, confounding changes coinciding with caching adoption (e.g., code growth or team changes) may affect our RQ_3 estimates, though we partially account for this by analyzing concurrent changes in code size, build jobs, and test density.

External Validity Our study analyzes builds from 1,279 GitHub projects on Travis CI, spanning up to six years and including many builds. Though the build data is from 2016, we confirm in 2021 that all projects remain active and maintained, and we successfully engage developers from 78% of contacted projects. Yet, results may not generalize to other CI services, such as CircleCI or GitHub Actions, which use different caching mechanisms and configuration rules that can cause different caching behaviors, misuses, or build issues. Future work should replicate and extend our study on modern CI services such as GitHub Actions and CircleCI, where caching mechanisms, default behaviors, and developer practices may differ substantially from Travis CI as of 2016.

7 Related Work

Studies on CI build durations. Prior work has extensively examined CI build durations. Rogers [47] showed that long builds disrupt development. Although ten minutes is often considered acceptable [9], only 16% of CI builds meet this threshold [18, 23], and long executions frequently result in timeouts [63]. To mitigate long builds, researchers proposed less frequent integration [47], decomposing large builds [3], and removing unnecessary dependencies or tests [31]. CI caching is a promising way to reduce build time [23, 26], and caching images and dependencies can further accelerate builds [20], while developers also apply custom acceleration strategies beyond caching [65]. Soares et al. [52] reviewed CI's effects, and Zheng et al. [68] analyzed CI workflow failures. Yet, the actual impact of caching on CI builds remains unclear and is likely context-dependent. Our study examines current CI caching practices and their associated performance improvements or overhead.

Studies on CI configuration. Several studies have explored CI configuration practices. Hilton et al. [31] reported that configuring CI builds is challenging, and Widder et al. [64] found that developers may abandon CI due to long builds, hard-to-diagnose failures [25], and configuration complexity [1, 21]. Vassallo et al. [62] identified smells in CI configurations. Recent work has analyzed CI configuration and testing practices in mobile apps [21, 44, 69], including workflow failures [68], automation and reuse [15], and the use of multiple CI services [11, 48]. Studies on GitHub Actions have examined CI workflow maintenance [58], complexity and compliance [1], developers' perceptions [51], security issues [36], and outdated actions [14]. Despite this, CI caching practices remain understudied. We address this gap by analyzing their adoption, maintenance, and impact on build performance.

Studies on CI build issues. Prior work examined CI build issues. Hilton et al. [31] studied assurance, security, and flexibility trade-offs, and Pinto et al. [46] analyzed developers' uncertainties and confusion. CI issues and bad practices have also been studied using Stack Overflow data [43, 66]. Tools such as CI-Odor [61] and BuildSonic [67] detect CI anti-patterns and Travis CI configuration smells. Khatami et al. [35] cataloged recurring anti-patterns in GitHub Actions workflows. However, CI caching issues remain largely underexplored. Our study investigates the challenges developers face when adopting CI caching and how to address them.

8 Conclusion

This paper presents an empirical study of CI caching practices in Travis CI, analyzing 513,384 builds from 1,279 GitHub projects to examine adoption, maintenance, impact on build duration, and common issues. Our results show that 70% of projects do not adopt CI caching, primarily due to limited awareness and varying project characteristics, and 76% of projects do not maintain their caches, highlighting that effective caching requires ongoing updates. We find that two-thirds of projects experience no significant reduction in build duration after enabling caching, indicating that benefits depend on proper maintenance. Moreover, two-thirds of projects (67%) cache all dependencies, which can lead to unnecessary cache uploads when artifacts change frequently, and caching may also introduce issues such as cache corruption or limited support for certain scenarios. These findings underscore the need for researchers, tool builders, and developers to collaborate on reducing CI cache maintenance overhead and improving caching support.

Future work. We plan to survey and interview developers to better understand awareness and practices related to CI caching, including the security implications of stale cached dependencies and the automated detection of optimal artifacts to cache based on project context and build history. We also plan to replicate our study on modern CI services such as GitHub Actions and CircleCI to assess whether our findings generalize beyond Travis CI.

References

[1] Edward Abrokwah and Taher A Ghaleb. 2025. An Empirical Study of Complexity, Heterogeneity, and Compliance of GitHub Actions Workflows. *arXiv preprint arXiv:2507.18062* (2025).

[2] Henri Aïdasso, Mohammed Sayagh, and Francis Bordeleau. 2025. Build optimization: A systematic literature review. *Comput. Surveys* 58, 1 (2025), 1–38.

[3] Glenn Ammons. 2006. Grexmk: speeding up scripted builds. In *Proceedings of the international workshop on Dynamic Systems Analysis*. ACM, 81–87.

[4] Marcus Emmanuel Barnes, Taher A Ghaleb, and Safwat Hassan. 2026. LogSieve: Task-Aware CI Log Reduction for Sustainable LLM-Based Analysis. In *Proceedings of the 23rd International Conference on Mining Software Repositories*. ACM, 1–12.

[5] Moritz Beller, Radjino Bholanath, Shane McIntosh, and Andy Zaidman. 2016. Analyzing the state of static analysis: A large-scale evaluation in open source software. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. IEEE, 470–481.

[6] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2017. Oops, my tests broke the build: An explorative analysis of Travis CI with GitHub. In *Proceedings of the 14th International Conference on Mining Software Repositories*. 356–367.

[7] Moritz Beller, Georgios Gousios, and Andy Zaidman. 2017. Travistorrent: Synthesizing travis ci and github for full-stack research on continuous integration. In *Proceedings of the 14th International Conference on Mining Software Repositories*. 447–450.

[8] Islem Bouzenia and Michael Pradel. 2024. Resource usage and optimization opportunities in workflows of GitHub Actions. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.

[9] Graham Brooks. 2008. Team pace keeping build times down. In *Proceedings of the AGILE Conference*. IEEE, 294–297.

[10] John L Campbell, Charles Quincy, Jordan Osserman, and Ove K Pedersen. 2013. Coding in-depth semistructured interviews: Problems of unitization and intercoder reliability and agreement. *Sociological Methods & Research* 42, 3 (2013), 294–320.

[11] Nitika Chopra and Taher A Ghaleb. 2025. From First Use to Final Commit: Studying the Evolution of Multi-CI Service Adoption. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 773–778.

[12] Thomas D Cook, Donald Thomas Campbell, and Arles Day. 1979. *Quasi-experimentation: Design & analysis issues for field settings*. Vol. 351. Houghton Mifflin Boston.

[13] Juliet M Corbin and Anselm Strauss. 1990. Grounded theory research: Procedures, canons, and evaluative criteria. *Qualitative sociology* 13, 1 (1990), 3–21.

[14] Alexandre Decan, Tom Mens, and Hassan Onsori Delicheh. 2023. On the outdatedness of workflows in the GitHub Actions ecosystem. *Journal of Systems and Software* 206 (2023), 111827.

[15] Hassan Onsori Delicheh, Guillaume Cardoen, Alexandre Decan, and Tom Mens. 2026. Automation and Reuse Practices in GitHub Actions Workflows: A Practitioner's Perspective. *arXiv preprint arXiv:2601.11299* (2026).

[16] Pedro Domingos. 2012. A few useful things to know about machine learning. *Commun. ACM* 55, 10 (2012), 78–87.

[17] Hamed Esfahani, Jonas Fietz, Qi Ke, Alexei Kolomiets, Erica Lan, Erik Mavrinac, Wolfram Schulte, Newton Sanches, and Srikanth Kandula. 2016. CloudBuild: Microsoft's distributed and caching build service. In *Proceedings of the 38th International Conference on Software Engineering Companion*. 11–20.

[18] Wagner Felidré, Leonardo Furtado, Daniel da Costa, Bruno Cartaxo, and Gustavo Pinto. 2019. Continuous Integration Theater. In *Proceedings of the 13th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 1–10.

[19] Martin Fowler. 2006. Continuous integration. https://www.martinfowler.com/articles/continuousIntegration.html Visited on January 20, 2021.

[20] Keheliya Gallaba, Yves Junqueira, John Ewart, and Shane Mcintosh. 2020. Accelerating Continuous Integration by Caching Environments and Inferring Dependencies. *IEEE Transactions on Software Engineering* (2020).

[21] Taher Ghaleb, Osamah Abduljalil, and Safwat Hassan. 2024. CI/CD Configuration Practices in Open-Source Android Apps: An Empirical Study. *ACM Transactions on Software Engineering and Methodology* (2024).

[22] Taher A Ghaleb. 2026. When AI Agents Touch CI/CD Configurations: Frequency and Success. In *Proceedings of the 23rd International Conference on Mining Software Repositories*. ACM, 1–5.

[23] Taher Ahmed Ghaleb, Daniel Alencar da Costa, and Ying Zou. 2019. An empirical study of the long duration of continuous integration builds. *Empirical Software Engineering* 24, 4 (2019), 2102–2139.

[24] Taher A. Ghaleb, Daniel Alencar da Costa, and Ying Zou. 2025. The promise and reality of continuous integration caching: an empirical study of Travis CI builds (Replication Package). https://github.com/Taher-Ghaleb/CICaching-EASE2026.

[25] Taher Ahmed Ghaleb, Daniel Alencar da Costa, Ying Zou, and Ahmed E Hassan. 2019. Studying the Impact of Noises in Build Breakage Data. *IEEE Transactions on Software Engineering* (2019), 1–14. doi:10.1109/TSE.2019.2941880

[26] Taher A Ghaleb, Safwat Hassan, and Ying Zou. 2022. Studying the interplay between the durations and breakages of continuous integration builds. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2476–2497.

[27] Taher A Ghaleb and Dulina Rathnayake. 2025. Can LLMs Write CI? A Study on Automatic Generation of GitHub Actions Configurations. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 767–772.

[28] GitHub. 2025. Migrating from Travis CI to GitHub Actions. https://docs.github.com/en/actions/migrating-to-github-actions/manually-migrating-to-github-actions/migrating-from-travis-ci-to-github-actions Accessed: March 14, 2025.

[29] Christian W Günther and Wil MP Van Der Aalst. 2007. Fuzzy mining–adaptive process simplification based on multi-perspective metrics. In *International conference on business process management*. Springer, 328–343.

[30] Frank E Harrell. 2001. Regression modeling strategies, with applications to linear models, survival analysis and logistic regression. *New York, NY: Springer* (2001).

[31] Michael Hilton, Nicholas Nelson, Timothy Tunnell, Darko Marinov, and Danny Dig. 2017. Trade-offs in continuous integration: assurance, security, and flexibility. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering*. ACM, 197–207.

[32] Md Nazmul Hossain and Taher A Ghaleb. 2025. CIgrate: Automating CI Service Migration with Large Language Models. *arXiv preprint arXiv:2507.20402* (2025).

[33] Guido W Imbens and Thomas Lemieux. 2008. Regression discontinuity designs: A guide to practice. *Journal of econometrics* 142, 2 (2008), 615–635.

[34] Frank E. Harrell Jr. 2025. rms: Regression Modeling Strategies. https://CRAN.R-project.org/package=rms R package version 8.

[35] Ali Khatami, Cćdric Willekens, and Andy Zaidman. 2024. Catching smells in the act: A GitHub Actions workflow investigation. In *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 47–58.

[36] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the security of github CI workflows. In *31st USENIX Security Symposium (USENIX Security 22)*. 2747–2763.

[37] Bogdan Korel. 1990. Automated software test data generation. *IEEE Transactions on software engineering* 16, 8 (1990), 870–879.

[38] Raula Gaikovina Kula, Daniel M German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. 2018. Do developers update their library dependencies? *Empirical Software Engineering* 23, 1 (2018), 384–417.

[39] Yang Luo, Yangyang Zhao, Wanwangying Ma, and Lin Chen. 2017. What are the Factors Impacting Build Breakage?. In *Proceedings of the 14th Conference on Web Information Systems and Applications Conference*. IEEE, 139–142.

[40] Shane McIntosh, Bram Adams, Thanh HD Nguyen, Yasutaka Kamei, and Ahmed E Hassan. 2011. An empirical study of build maintenance effort. In *2011 33rd International Conference on Software Engineering (ICSE)*. IEEE, 141–150.

[41] Ansong Ni and Ming Li. 2017. Cost-effective build outcome prediction using cascaded classifiers. In *Proceedings of the 14th International Conference on Mining Software Repositories*. 455–458.

[42] Johannes Nicolai. [n. d.]. GitHub welcomes all CI tools. https://github.blog/2017-11-07-github-welcomes-all-ci-tools Visited on January 20, 2021.

[43] Ali Ouni, Islem Saidani, Eman Alomar, and Mohamed Wiem Mkaouer. 2023. An empirical study on continuous integration trends, topics and challenges in Stack Overflow. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. 141–151.

[44] Hamid Parsazadeh, Taher A Ghaleb, and Safwat Hassan. 2026. Android Instrumentation Testing in Continuous Integration: Practices, Patterns, and Performance. In *Proceedings of the 19th IEEE International Conference on Software Testing, Verification and Validation*. ACM, 1–12.

[45] Ivan Pashchenko, Henrik Plate, Serena Elisa Ponta, Antonino Sabetta, and Fabio Massacci. 2018. Vulnerable open source dependencies: Counting those that matter. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 1–10.

[46] Gustavo Pinto, Fernando Castor, Rodrigo Bonifacio, and Marcel Rebouças. 2018. Work practices and challenges in continuous integration: A survey with Travis CI users. *Software: Practice and Experience* 48, 12 (2018), 2223–2236.

[47] R Owen Rogers. 2004. Scaling continuous integration. In *International Conference on Extreme Programming and Agile Processes in Software Engineering*. Springer, 68–76.

[48] Pooya Rostami Mazrae, Tom Mens, Mehdi Golzadeh, and Alexandre Decan. 2023. On the usage, co-usage and migration of CI/CD tools: A qualitative analysis. *Empirical Software Engineering* 28, 2 (2023), 52.

[49] Jadson Santos, Daniel Alencar da Costa, Shane McIntosh, and Uirá Kulesza. 2025. On the need to monitor continuous integration practices. *Empirical Software Engineering* 30, 5 (2025), 125.

[50] WS Sarle. 1990. The VARCLUS Procedure. *SAS/STAT User's Guide* (1990).

[51] Sk Golam Saroar and Maleknaz Nayebi. 2023. Developers' perception of GitHub Actions: A survey analysis. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*. 121–130.

[52] Eliezio Soares, Gustavo Sizilio, Jadson Santos, Daniel Alencar Da Costa, and Uirá Kulesza. 2022. The effects of continuous integration on software development: a systematic literature review. *Empirical Software Engineering* 27, 3 (2022), 78.

[53] Travis CI. [n. d.]. Speeding up the build. https://docs.travis-ci.com/user/speeding-up-the-build Visited on January 05, 2021.

[54] Travis CI. 2013. Speed up your builds: cache your dependencies. https://web.archive.org/web/20160304061044/http://blog.travis-ci.com/2013-12-05-speed-up-your-builds-cache-your-dependencies/

[55] Travis CI. 2014. Faster Builds with Container-Based Infrastructure and Docker. https://web.archive.org/web/20190825085224/https://blog.travis-ci.com/2014-12-17-faster-builds-with-container-based-infrastructure

[56] Travis CI. 2016. Caching now available for everyone. https://web.archive.org/web/20190510075306/https://blog.travis-ci.com/2016-05-03-caches-are-coming-to-everyone

[57] Travis CI. 2025. Caching Dependencies and Directories. https://docs.travis-ci.com/user/caching Accessed: August 13, 2025.

[58] Pablo Valenzuela-Toledo, Alexandre Bergel, Timo Kehrer, and Oscar Nierstrasz. 2024. The hidden costs of automation: An empirical study on GitHub Actions workflow maintenance. In *2024 IEEE International Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 213–223.

[59] Boudewijn F Van Dongen, Ana Karla A de Medeiros, HMW Verbeek, AJMM Weijters, and Wil MP van Der Aalst. 2005. The ProM framework: A new era in process mining tool support. In *International conference on application and theory of petri nets*. Springer, 444–454.

[60] Joachim Vandekerckhove, Dora Matzke, and Eric-Jan Wagenmakers. 2015. Model Comparison and the Principle. In *The Oxford handbook of computational and mathematical psychology*. Vol. 300. Oxford Library of Psychology.

[61] Carmine Vassallo, Sebastian Proksch, Harald C Gall, and Massimiliano Di Penta. 2019. Automated reporting of anti-patterns and decay in continuous integration. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 105–115.

[62] Carmine Vassallo, Sebastian Proksch, Anna Jancso, Harald C Gall, and Massimiliano Di Penta. 2020. Configuration smells in continuous delivery pipelines: a linter and a six-month study on GitLab. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 327–337.

[63] Nimmi Weeraddana, Mahmoud Alfadel, and Shane McIntosh. 2024. Characterizing timeout builds in continuous integration. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1450–1463.

[64] David Gray Widder, Michael Hilton, Christian Kästner, and Bogdan Vasilescu. 2019. A conceptual replication of continuous integration pain points in the context of Travis CI. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 647–658.

[65] Mingyang Yin, Yutaro Kashiwa, Keheliya Gallaba, Mahmoud Alfadel, Yasutaka Kamei, and Shane McIntosh. 2024. Developer-applied accelerations in continuous integration: A detection approach and catalog of patterns. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1655–1666.

[66] Fiorella Zampetti, Carmine Vassallo, Sebastiano Panichella, Gerardo Canfora, Harald Gall, and Massimiliano Di Penta. 2020. An empirical characterization of bad practices in continuous integration. *Empirical Software Engineering* 25, 2 (2020), 1095–1135.

[67] Chen Zhang, Bihuan Chen, Junhao Hu, Xin Peng, and Wenyun Zhao. 2022. BuildSonic: Detecting and repairing performance-related configuration smells for continuous integration builds. In *Proceedings of the 37th IEEE/ACM international conference on automated software engineering*. 1–13.

[68] Lianyu Zheng, Shuang Li, Xi Huang, Jiangnan Huang, Bin Lin, Jinfu Chen, and Jifeng Xuan. 2025. Why Do GitHub Actions Workflows Fail? An Empirical Study. *ACM Transactions on Software Engineering and Methodology* (2025).

[69] Xiaoxin Zhou, Taher A Ghaleb, and Safwat Hassan. 2026. Role of CI Adoption in Mobile App Success: An Empirical Study of Open-Source Android Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*. ACM, 1–12.